EXPERIMENT NO. 07

Date of Performance:
Date of Submission:

1. **AIM:** Build a program to illustrate the concept of distributed deadlock detection

1.1.OBJECTIVES

1. To understand the concept of deadlock in distributed systems.
2. To study how processes compete for shared resources in a distributed environment.
3. To implement a program that simulates resource allocation and process dependency.
4. To detect deadlock situations using a suitable detection technique (such as Chandy-Misra-Haas's Distributed Deadlock Detection Algorithm).
5. To analyze and display the processes involved in the deadlock.
6. To gain practical understanding of deadlock detection mechanisms in distributed computing.

1.2. SCOPE

Understanding Deadlock and Resource Competition in Distributed Systems

This experiment focuses on understanding how deadlocks occur in distributed systems when multiple processes compete for shared resources. It explains how improper allocation of resources and circular waiting conditions can lead to a deadlock situation where processes are unable to proceed.

Simulation of Process Dependency and Resource Allocation

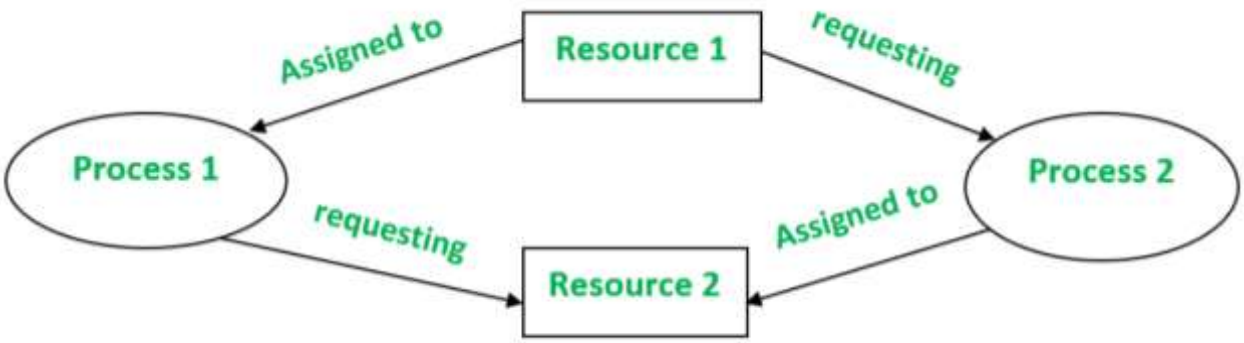
The scope also includes simulating the interaction between processes and resources in a distributed environment. The program will demonstrate how processes request and hold resources, and how dependencies between processes are formed during execution.

Detection and Analysis of Deadlock Conditions

Another important scope of the experiment is to detect deadlock situations using an appropriate detection mechanism such as a wait-for graph or dependency graph. The program will help in identifying the processes involved in the deadlock and analyzing the system state when the deadlock occurs.

2. LITERATURE SURVEY

A Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource occupied by some other process. When this situation arises, it is known as Deadlock.



What are Deadlocks in Distributed Systems?

In distributed systems, a deadlock occurs when a set of processes are unable to proceed because each is waiting for a resource that another holds, creating a circular wait condition. This situation causes the involved processes to become stuck, unable to continue or complete their tasks. Key Characteristics of Deadlocks include:

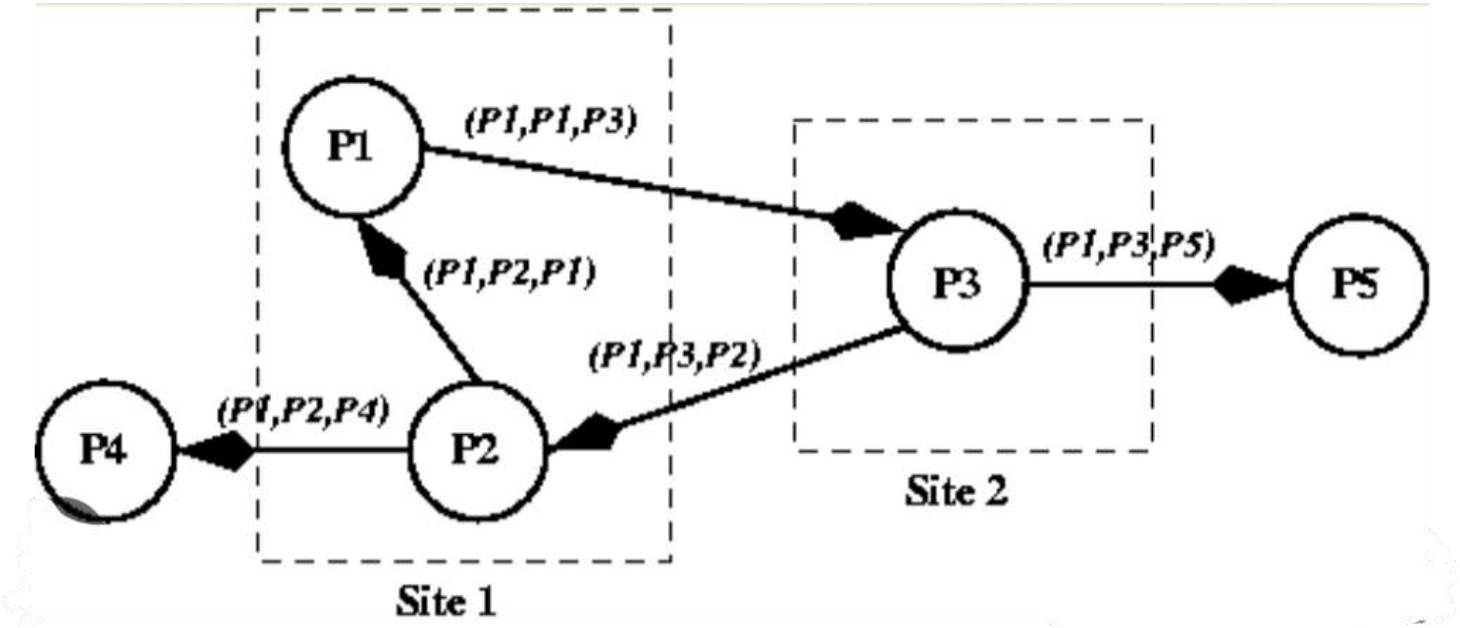
1. **Mutual Exclusion**: At least one resource is held in a non-shareable mode, meaning only one process can use it at a time.
2. **Hold and Wait**: Processes holding resources can request additional resources without releasing their current ones.
3. **No Preemption**: Resources cannot be forcibly taken from a process; they must be released voluntarily.
4. **Circular Wait**: A closed loop of processes exists where each process is waiting for a resource held by the next process in the loop.

Three commonly used strategies to handle deadlocks are as follows

1. **Avoidance:** Resources are carefully allocated to avoid deadlocks.
2. **Prevention:** Constraints are imposed on the ways in which processes request resources in order to prevent deadlocks.
3. **Detection and recovery:** Deadlocks are allowed to occur and a detection algorithm is used to detect them. After a deadlock is detected, it is resolved by certain means.

2.1.Chandy-Misra-Haas's Distributed Deadlock Detection Algorithm

Chandy-Misra-Haas's distributed deadlock detection algorithm is an edge chasing algorithm to detect deadlock in distributed systems.
In edge chasing algorithm, a special message called *probe* is used in deadlock detection. A *probe* is a triplet *(i, j, k)* which denotes that process Pi has initiated the deadlock detection and the message is being sent by the home site of process Pj to the home site of process Pk.
The probe message circulates along the edges of WFG to detect a cycle. When a blocked process receives the probe message, it forwards the probe message along its outgoing edges in WFG. A process Pi declares the deadlock if probe messages initiated by process Pi returns to itself.



Other terminologies used in the algorithm

1. **Dependent process:**
   A process Pi is said to be dependent on some other process Pj, if there exists a sequence of processes Pi, Pi1, Pi2, Pi3..., Pim, Pj such that in the sequence, each process except Pj is blocked and each process except Pi holds a resource for which previous process in the sequence is waiting.
2. **Locally dependent process:**
   A process Pi is said to be locally dependent on some other process Pj if the process Pi is dependent on process Pj and both are at same site.

Data structures

A boolean array, **dependenti**. Initially, **dependenti[j]** is false for all value of i and j. **dependenti[j]** is true if process Pj is dependent on process Pi.

2.2. Algorithm

In psuedocode, the algorithm works as follows:

Controller sending a probe

if P_j is locally dependent on itself
  then declare deadlock
else for all P_j,P_k such that
  (i) P_i is locally dependent on P_j,
  (ii) P_j is waiting for 'P_k and
  (iii) P_j, P_k are on different controllers.
send probe(i, j, k). to home site of P_k

Controller receiving a probe

if
  (i)P_k is idle / blocked
  (ii) $dependent_k$(i) = false, and
  (iii) P_k has not replied to all requests of to P_j
then begin
  "dependents"$_{"k"}$(i) = true;
  if k == i
  then declare that P_i is deadlocked
  else for all P_a,P_b such that
    (i) P_k is locally dependent on P_a,
    (ii) P_a is waiting for 'P_b and
    (iii) P_a, P_b are on different controllers.
  send probe(i, a, b). to home site of P_b
end

2.3.Sample Input Examples

Processes: P1, P2, P3
Sites: P1 → S1, P2 → S2, P3 → S1
Dependencies:
P1 → P2
P2 → P3
P3 → P1

Output

Deadlock detected involving P1

Performance

Algorithm requires at most exchange of m(n-1)/2 messages to detect deadlock. Here, m is number of processes and n is the number of sites.
The delay in detecting the deadlock is O(n).

2.4. Programming Language Used

C Programming Language

2.4.1. Libraries Used

1. stdio.h
It used to read input values (number of processes, dependencies, initiator process) and to display probe messages and deadlock results.
Syntax:
#include <stdio.h>

2.4.2. Functions Used

1. main()
It is the starting point of the program. It reads the number of processes, wait-for dependencies, and the initiator process from the user and starts the deadlock detection process.
Syntax:
int main()

2. printf()
It is used to display output messages such as probe messages and deadlock detection results.
Syntax:
printf("message");

3. scanf()
It is used to read input values from the user.
Syntax:
scanf("format_specifier", &variable);

4. addEdge()
It is used to add a dependency between two processes in the wait-for graph.
Syntax:
void addEdge(int u, int v)

5. detectDeadlock()
It is used to send probe messages and check whether the probe returns to the initiator, indicating a deadlock.
Syntax:
int detectDeadlock(int initiator)

3. IMPLEMENTATION

3.1.Program

```
#include <stdio.h>
#define MAX 20
int waitfor[MAX][MAX];
int n;
/* add dependency Pi -> Pj */
void addEdge(int u,int v)
{
   waitfor[u][v]=1;
}
int detectDeadlock(int initiator)
{
   int queue_i[100],queue_j[100],queue_k[100];
   int front=0,rear=0;
   /* initial probes */
   for(int p=1;p<=n;p++)
   {
      if(waitfor[initiator][p])
      {
         queue_i[rear]=initiator;
         queue_j[rear]=initiator;
         queue_k[rear]=p;
         rear++;
      }
   }
```

```
    while(front<rear)
    {
        int i=queue_i[front];
        int j=queue_j[front];
        int k=queue_k[front];
        front++;
        printf("Probe sent: (%d,%d,%d)\n",i,j,k);
        if(k==i)
        return 1;
        for(int m=1;m<=n;m++)
        {
            if(waitfor[k][m])
            {
                printf("Process %d forwards probe to Process %d\n",k,m);
                queue_i[rear]=i;
                queue_j[rear]=k;
                queue_k[rear]=m;
                rear++;
            }
        }
    }
    return 0;
}
int main()
{
    int e,u,v,initiator;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    printf("Enter number of wait-for dependencies: ");
    scanf("%d",&e);
    printf("\nEnter dependencies (Pi waiting for Pj):\n");
    for(int i=0;i<e;i++)
    {
        printf("Process Pi: ");
        scanf("%d",&u);
        printf("Waiting for Pj: ");
        scanf("%d",&v);
        addEdge(u,v);
    }
    printf("\nEnter initiator process: ");
    scanf("%d",&initiator);
    if(detectDeadlock(initiator))
        printf("\nDeadlock Detected!\n");
    else
        printf("\nNo Deadlock Detected\n");
    return 0;
}
```

3.2. **Output**

Command: gcc exp.c

```
Enter number of processes: 3
Enter number of wait-for dependencies: 3

Enter dependencies (Pi waiting for Pj):
Process Pi: 1
Waiting for Pj: 2
Process Pi: 2
Waiting for Pj: 3
Process Pi: 3
Waiting for Pj: 1

Enter initiator process: 1
Probe sent: (1,1,2)
Process 2 forwards probe to Process 3
Probe sent: (1,2,3)
Process 3 forwards probe to Process 1
Probe sent: (1,3,1)

Deadlock Detected!
```

CONCLUSION

In this experiment, a program was developed to illustrate the concept of distributed deadlock detection using the Chandy–Misra–Haas Distributed Deadlock Detection Algorithm in Distributed Systems. The experiment helped in understanding how deadlocks occur when multiple processes compete for shared resources in a distributed environment. By simulating process dependencies through a wait-for graph and propagating probe messages among processes, the program was able to detect circular waiting conditions and identify the processes involved in a deadlock. This experiment provided practical knowledge about how deadlock detection mechanisms work in distributed systems and highlighted the importance of proper resource management to avoid system blocking and ensure efficient execution of processes.

SIGN AND REMARKS:

R1 (4 Marks)	R2 (4 Marks)	R3 (4 Marks)	R4 (3 Marks)	Total Marks (15 Marks)	Signature